

Sparse, Low-Rank, and Deep

A Complete Walkthrough and Presenter's Companion

for the ENGS 109 final project on music source separation

Taka Khoo

May 31, 2026

What this document is. A long, plain-English companion to the paper and the poster. It explains, one piece at a time, *what* the project does, *why* each choice was made, and *how* to talk about every table, figure, and diagram. Boxes mark three kinds of help: an **Analogy** for intuition, a **Say this** script you can read almost verbatim at the poster, and **The math, slowly** for when someone wants the real equations. Read it start to finish to understand the work, or jump to Part 6 (the poster walk) and Part 8 (Q&A) the night before you present.

Contents

1	The one-paragraph version	2
2	Part 1: The problem, from scratch	2
2.1	What “source separation” means	2
2.2	Why it is genuinely hard	2
2.3	The spectrogram: a picture of sound	3
2.4	Two magic words: sparse and low-rank	3
3	Part 2: The theory that licenses everything	3
3.1	Why music sits comfortably inside the theory	4
4	Part 3: The four separators, one at a time	4
4.1	Method A — Robust PCA (the low-rank-plus-sparse trick)	5
4.2	Method B — Supervised NMF (learning ingredient templates)	5
4.3	Method C — K-SVD plus SRC (a custom alphabet per source)	6
4.4	Method D — the deep sparse coder (the new contribution)	6
4.5	Method E — stereo as joint sparsity	7
5	Part 4: Every experiment, explained	7
5.1	How we score: SI-SDR and the decibel scale	7
5.2	The headline table, row by row	7
5.3	The trivial floors (and why they matter)	8
5.4	The ablations: what each knob does	8
5.5	Soft versus binary masks	8
5.6	Mask fusion	8
5.7	Stereo	8
5.8	The coherence “negative result” (and why we are proud of it)	8

6	Part 5: Every figure, explained	9
7	Part 6: Walking the poster, block by block	10
8	Part 7: Walking the paper, section by section	10
9	Part 8: Questions you will get, and answers	11
10	Part 9: Glossary	12
11	Part 10: References and where to look	13

1 The one-paragraph version

We take a finished song and pull it back apart into two tracks: the **vocals** and the **accompaniment** (everything else). We do this four different ways, and the punchline is that all four are secretly the same idea: find a *structured* (simple, patterned) description of the sound and use it to decide which part of each instant belongs to the voice. The four ways climb a ladder of sophistication, from a training-free matrix trick, to learned “dictionaries” of sound, to a small neural network that is really a classical solver in disguise. The neural network wins, but the real contribution is showing they are one family and measuring exactly how far the textbook theory carries into practice.

Say this (script)

“Everyone has heard a karaoke track or an a-cappella remix. That is source separation: given the final mix, recover the hidden tracks. I built four separators that look different but are all doing one thing, recovering a *structured* version of the spectrogram, and I show that the same compressed sensing math from this course explains all of them. The deep-learning one wins, but it is literally a classical sparse-recovery solver with its steps unrolled into layers.”

2 Part 1: The problem, from scratch

2.1 What “source separation” means

A recording engineer takes many tracks, vocals, drums, bass, guitar, and sums them into one stereo file. **Source separation** is the inverse: given only that final file, recover the original tracks. We focus on the two-way split, voice versus everything else, because it is the most useful and the cleanest to measure.

Analogy

Think of mixing as pouring several colors of paint into one bucket and stirring. Source separation is being handed the bucket and asked for the original colors back. In general that is impossible, infinitely many color sets stir to the same brown, which is exactly why we need a **prior**: an assumption about what real tracks look like that rules out most of the bad answers.

2.2 Why it is genuinely hard

Summation destroys information. If $a + b = 10$, you cannot recover a and b without more knowledge. The whole game is supplying that extra knowledge in the form of structure: “a singing voice looks like *this*, a backing track looks like *that*.” Every method in this project is a different way of writing down that “looks like.”

2.3 The spectrogram: a picture of sound

We never work on the raw audio waveform. We first run the **short-time Fourier transform (STFT)**, which chops the audio into short overlapping windows and, for each window, measures how much energy sits at each frequency. The result is an image: time runs left to right, pitch runs bottom (low) to top (high), and brightness is loudness. This image is called the **magnitude spectrogram**.

Analogy

A spectrogram is sheet music drawn by a machine. A low bass note is a bright stripe near the bottom; a high cymbal is a fuzzy cloud near the top; a sung vowel is a set of evenly spaced horizontal lines (the voice’s harmonics). Once sound is a picture, “separate the voice” becomes “figure out which pixels belong to the voice,” and that is an image problem we have tools for.

The math, slowly

The STFT of a signal $y(t)$ is a complex matrix $\mathbf{Y} \in \mathbb{C}^{F \times T}$: F frequency bins, T time frames. We split it into a nonnegative **magnitude** $\mathbf{M} = |\mathbf{Y}|$ (how loud) and a **phase** $\angle \mathbf{Y}$ (the waveform’s alignment). The transform is invertible: given magnitude and phase we can rebuild the exact audio. All our methods edit the magnitude and borrow the original mixture’s phase to resynthesize.

2.4 Two magic words: sparse and low-rank

Everything rests on two observed regularities.

- **The voice is sparse.** At any instant, a sung note lights up only a handful of harmonic lines; the rest of that column of the image is essentially black. “Sparse” just means “mostly zeros, few nonzeros.”
- **The accompaniment is low-rank.** Backing music repeats: the same chord, the same drum loop, again and again. A spectrogram built from a few repeating patterns is “low-rank,” meaning it can be written as a short sum of repeated building blocks.

Analogy

Sparse is a night sky: almost all black, a few bright stars. **Low-rank** is wallpaper: a small tile repeated across a whole wall. The voice is the stars, the band is the wallpaper, and that difference is the wedge we drive between them.

Why this matters for the whole project: “sparse” and “low-rank” are exactly the two structures that **compressed sensing**, the subject of this course, is built to recover. So music separation is not a new problem requiring new math; it is a textbook compressed-sensing problem wearing headphones.

3 Part 2: The theory that licenses everything

Compressed sensing answers a surprising question: if a signal is sparse, can you recover it from far fewer measurements than its length? The answer is yes, provided the measurement system is “spread out” enough. Three conditions certify success.

The math, slowly

We have $\mathbf{y} = \mathbf{A}\mathbf{x}$ with $\mathbf{x}$ *s-sparse* (at most s nonzeros) and $\mathbf{A}$ short and wide (fewer rows than columns, so the system is underdetermined). The honest objective is to find the sparsest solution, $\min \|\mathbf{z}\|_0$ such that $\mathbf{A}\mathbf{z} = \mathbf{y}$, but counting nonzeros (ℓ_0) is NP-hard. We relax to the ℓ_1 norm, $\min \|\mathbf{z}\|_1$ s.t. $\mathbf{A}\mathbf{z} = \mathbf{y}$, a linear program. Three sufficient conditions guarantee the relaxation returns the true sparse $\mathbf{x}$:

$$\underbrace{\text{spark}(\mathbf{A}) > 2s}_{\text{uniqueness}}, \quad \underbrace{\delta_{2s} < \sqrt{2} - 1}_{\text{RIP}}, \quad \underbrace{s < \frac{1}{2} \left(1 + \frac{1}{\mu(\mathbf{A})}\right)}_{\text{coherence}}.$$

Spark is the smallest number of columns of $\mathbf{A}$ that are linearly dependent; if it exceeds $2s$, no two s -sparse signals collide. **RIP** (the restricted isometry property) says every small group of columns nearly preserves length, $(1 - \delta_s)\|\mathbf{x}\|_2^2 \leq \|\mathbf{A}\mathbf{x}\|_2^2 \leq (1 + \delta_s)\|\mathbf{x}\|_2^2$. **Coherence** $\mu(\mathbf{A}) = \max_{i \neq j} |\langle \mathbf{a}_i, \mathbf{a}_j \rangle|$ is the largest similarity between two columns; small coherence means the building blocks are distinguishable.

Analogy

Coherence is a line-up of suspects. If two suspects look almost identical (high coherence) a witness cannot tell them apart, and recovery is ambiguous. If everyone looks distinct (low coherence) one witness statement pins the culprit. The dictionary's "atoms" are the suspects; we want them to look different.

3.1 Why music sits comfortably inside the theory

Real numbers from a 6–8 second clip: the spectrogram is about 1025×340 ; a voiced frame uses roughly $s \approx 10$ atoms out of a $K = 96$ dictionary (about 10% active); the accompaniment's energy collapses into $r \approx 4$ –8 repeating components. Sparse voice plus low-rank band is precisely the "low-rank-plus-sparse" model that one of our methods (Robust PCA) provably separates.

Say this (script)

"The course proves you can recover a sparse signal from few measurements if the measurement matrix is incoherent enough. Music hands us that situation for free: the voice is about ten harmonics out of a hundred, and the band is a handful of repeating patterns. So I am not hoping the math applies, I can point to the exact sparsity and rank numbers that put us inside the recovery guarantees."

4 Part 3: The four separators, one at a time

All four produce a **mask**: a number between 0 and 1 for every cell of the spectrogram saying "what fraction of this cell is voice." Multiply the mixture by the mask, borrow the mixture's phase, invert the STFT, and you have an audio track.

Analogy

A mask is a dimmer switch on every pixel. The vocal mask turns the voice pixels up to 1 and the band pixels down to 0; the accompaniment mask does the opposite. A *soft* mask is a real dimmer (any value in between); a *binary* mask is a plain on/off switch. We will see the dimmer wins.

4.1 Method A — Robust PCA (the low-rank-plus-sparse trick)

Idea: split the magnitude spectrogram $\mathbf{M}$ into a low-rank part $\mathbf{X}$ (the repeating band) and a sparse part $\mathbf{E}$ (the voice), with no training at all.

The math, slowly

Solve the convex program (“principal component pursuit”):

$$\min_{\mathbf{X}, \mathbf{E}} \|\mathbf{X}\|_* + \lambda \|\mathbf{E}\|_1 \quad \text{s.t.} \quad \mathbf{X} + \mathbf{E} = \mathbf{M}, \quad \lambda = \frac{1}{\sqrt{\max(F, T)}}.$$

$\|\mathbf{X}\|_*$ is the **nuclear norm** (sum of singular values), the convex stand-in for “low rank”; $\|\mathbf{E}\|_1$ is the convex stand-in for “sparse.” The single knob λ is fixed by theory, no tuning. We solve it by the inexact augmented Lagrange multiplier method, which alternates a singular-value threshold (shrinks small singular values to zero, enforcing low rank) with a soft threshold (shrinks small entries to zero, enforcing sparsity).

Analogy

Imagine a security camera watching a lobby. The walls and furniture never move (low-rank background); a person walking through is the rare, moving part (sparse foreground). Robust PCA is the standard trick to pull the moving person out of a static scene. Here the “moving person” is the singer and the “static lobby” is the looping band.

How it does: weakest of the learned methods on voice (it has no idea what a voice actually looks like, only that it is the leftover sparse part), but a useful, training-free baseline that the others must beat.

4.2 Method B — Supervised NMF (learning ingredient templates)

Idea: before testing, learn a small **dictionary** of spectral templates for each source from isolated examples; at test time, explain the mixture as a nonnegative combination of those templates and read off how much belongs to each source.

The math, slowly

Non-negative matrix factorization approximates $\mathbf{M} \approx \mathbf{D}\mathbf{H}$ with $\mathbf{D}, \mathbf{H} \geq 0$: $\mathbf{D}$ holds spectral **atoms** (templates), $\mathbf{H}$ holds their time-varying **activations** (how loud each template is, frame by frame). We learn $\mathbf{D}_v$ (voice) and $\mathbf{D}_a$ (accompaniment) separately with the Lee–Seung multiplicative updates

$$\mathbf{H} \leftarrow \mathbf{H} \odot \frac{\mathbf{D}^\top (\mathbf{M} \oslash \mathbf{D}\mathbf{H})}{\mathbf{D}^\top \mathbf{1}}, \quad \mathbf{D} \leftarrow \mathbf{D} \odot \frac{(\mathbf{M} \oslash \mathbf{D}\mathbf{H})\mathbf{H}^\top}{\mathbf{1}\mathbf{H}^\top}.$$

At test time we freeze $\mathbf{D} = [\mathbf{D}_v \mid \mathbf{D}_a]$, solve only for $\mathbf{H}$, and the vocal magnitude estimate is $\mathbf{D}_v \mathbf{H}_v$. These updates are not a hack: they are a *majorization-minimization* scheme that provably never increases the error.

Analogy

A smoothie tastes like “banana + strawberry + a little spinach.” NMF learns the pure flavor of each fruit (the atoms), then, tasting a new smoothie (the mixture), estimates how much of each fruit is in it (the activations). Knowing the voice’s “pure flavors” lets us reconstruct just the voice.

4.3 Method C — K-SVD plus SRC (a custom alphabet per source)

Idea: learn a sharper, *overcomplete* dictionary (more atoms than dimensions) that can sparsely spell out each source, then classify each cell by which source’s alphabet spells it best.

The math, slowly

K-SVD learns $\mathbf{D}$ so that every training frame is an s -sparse combination of atoms ($\|\boldsymbol{\alpha}\|_0 \leq s$). It alternates (i) sparse coding with orthogonal matching pursuit (OMP) and (ii) updating one atom at a time by a rank-one SVD of the residual, which is *optimal* for that atom (Eckart–Young). At test time we sparse-code each mixture frame on the combined dictionary $[\mathbf{D}_v \mid \mathbf{D}_a]$ and assign energy by the sparse-representation-classification (SRC) rule: pick the source whose atoms reconstruct the frame with the smallest residual, $c^* = \arg \min_c \|\mathbf{y} - \mathbf{D}_c \boldsymbol{\alpha}_c\|_2$.

Analogy

Give each singer their own alphabet of sound-letters. A frame “written” cleanly in the voice alphabet but gibberish in the band alphabet is clearly voice. K-SVD designs those alphabets; SRC reads each frame in both and keeps whichever spelling is neater.

4.4 Method D — the deep sparse coder (the new contribution)

Idea: a neural network is really a sparse-recovery solver with its iterations *unrolled* into layers. Keep that structure but *learn* the dictionaries by backpropagation, so the network is both interpretable and trainable.

The math, slowly

Each module computes two stacked nonnegative sparse codes, a **fat** (upsampling) one then a **tall** (downsampling) one:

$$\boldsymbol{\alpha}^{(1)} = \arg \min_{\boldsymbol{\alpha} \geq 0} \frac{1}{2} \|\mathbf{x} - \mathbf{D}^{(1)} \boldsymbol{\alpha}\|_2^2 + \lambda_1 \|\boldsymbol{\alpha}\|_1 + \frac{\lambda_2}{2} \|\boldsymbol{\alpha}\|_2^2.$$

We solve it by K steps of a non-negative fast proximal-gradient iteration (FISTA), each step a matrix multiply, a gradient step, and a clamp:

$$\boldsymbol{\alpha}_k = \left[\mathbf{z}_k - \frac{1}{L} \mathbf{D}^\top (\mathbf{D} \mathbf{z}_k - \mathbf{x}) - \frac{\lambda_1}{L} \right]_+.$$

Stack four such modules, add a sigmoid head to emit the mask, feed each prediction a ± 2 -frame time window, and train end-to-end against the ideal mask. This is the LISTA construction (“learned ISTA”), specialized to non-negative codes and a masking output.

Analogy

Sparse coding is a slow recipe with many stirring steps. Unrolling is writing each stir as its own station on an assembly line; once it is an assembly line, you can *tune* each station for the best result instead of using the textbook setting. The network is that tuned assembly line. It is not a black box, every layer is a recognizable solver step.

This is the project’s novelty. The architecture (composite sparse-coding modules, fat-then-tall) was published only for image classification. Adapting it to predict an audio time-frequency mask, and showing it beats the classical methods, is the new result.

4.5 Method E — stereo as joint sparsity

Stereo has a left and a right channel that share the same active sources at each instant; only the balance differs. So their sparse codes share the same *support* (the same atoms are on). Simultaneous OMP recovers that shared support, which makes the problem easier (more measurements, same unknowns) and keeps the two channels consistent so the stereo image does not wobble.

5 Part 4: Every experiment, explained

5.1 How we score: SI-SDR and the decibel scale

We measure quality with **SI-SDR** (scale-invariant signal-to-distortion ratio), in decibels. Higher is better. It compares the recovered track to the true track after removing any overall volume difference (that is the “scale-invariant” part, it does not reward or punish loudness, only content).

Analogy

Decibels are a log scale, like earthquake magnitudes. Every +3 dB roughly halves the leftover error. So the jump from K-SVD’s 0.8 dB to the deep coder’s 2.0 dB on voice is not “a little better,” it is a clearly audible reduction in bleed.

Read accompaniment scores against a floor, not zero. Because the band dominates a mix’s energy, even “do nothing, return the mixture” scores about +4 dB on accompaniment. So +8.3 dB is not “twice as good as +4”; it is clearing a high floor. We list that floor explicitly so nobody is fooled.

5.2 The headline table, row by row

This is Table 2 of the paper. Read it top to bottom as a ladder.

Method	Voc. SI-SDR	Acc. SI-SDR	What it shows
Mixture (no split)	−3.7	4.0	the “do nothing” floor
Random mask	−3.9	2.9	the no-information floor
Low-rank (rank 4)	−0.1	6.1	low-rank alone helps the band
HPSS (median)	−11.9	−1.4	classical, over-suppresses voice
REPET-SIM	−0.7	2.9	classical, needs strict repetition
Robust PCA	−3.1	0.2	ours, training-free
Supervised NMF	0.5	5.1	ours, learned templates
K-SVD + SRC	0.8	7.1	ours, learned alphabets
Deep coder	2.0	8.3	ours, best on both
Oracle IRM/IBM	10.2	15.8	the best any mask can do

Say this (script)

“Read the table as a story. The top rows are floors: doing nothing already scores four on the band because the band is most of the energy. The classical baselines are mixed, one of them actually hurts. Then our methods climb: training-free, then learned templates, then learned alphabets, then the deep coder on top, two decibels on voice, eight on the band, the best non-oracle on both. The bottom row is the oracle, the answer key, which no real method can pass.”

5.3 The trivial floors (and why they matter)

Three “non-methods” anchor the scale: returning the mixture unchanged, a random mask, and a plain rank-4 truncation. They cost nothing to compute and they stop us from over-claiming: any real method has to clear these to mean anything. The mixture floor in particular is why we always quote the accompaniment number next to it.

5.4 The ablations: what each knob does

This is Table 3. An ablation changes one setting at a time and watches the score, so we learn *why* the method works, not just *that* it works.

- **NMF dictionary size K (16, 32, 64, 128).** Voice score is *highest at the smallest* dictionary ($K = 16$) and falls as K grows. With limited training data, a small dictionary is a stronger prior and generalizes better. This is a regularization effect, more atoms is not always better.
- **K-SVD sparsity s (3, 5, 10, 15).** Score climbs to $s = 10$ then flattens: richer codes help until extra atoms stop explaining anything new.
- **Robust PCA penalty λ (scaled 0.5 to 2.0).** A larger penalty pushes more energy into the low-rank band, leaving a cleaner sparse voice, so the textbook λ is actually conservative for this task.

Say this (script)

“The ablations are the honest part. The most interesting one is that smaller NMF dictionaries generalize *better* here, fewer building blocks act as a stronger prior when you do not have much training data. That is a real, slightly counter-intuitive finding, not a number I tuned to look good.”

5.5 Soft versus binary masks

Same models, two ways of applying the mask. The soft (dimmer) mask beats the binary (on/off) mask by about 1 dB for the deep coder (2.1 vs 1.0). Reason: many cells are genuinely shared between voice and band; a soft value keeps the right fraction, while a hard 0/1 decision throws away the overlap. This mirrors the gap between the two oracles (ratio mask vs binary mask) and justifies training toward a soft target.

5.6 Mask fusion

We also averaged the three learned masks (an ensemble). Earlier, when the deep model was weaker, fusion helped the band slightly. Now that the deep model is strong, fusion ties it on voice and no longer beats it on the band, so we report the single model as the headline and say so. Reporting this honestly (rather than quietly keeping fusion when it stopped helping) is the point.

5.7 Stereo

Run on 10 stereo tracks, the joint-sparsity method matches the mono K-SVD dictionary it is built on (0.9 dB voice, 7.3 dB band) while keeping the two channels consistent. It is a proof of concept that the multi-measurement idea works on real stereo, not a headline number.

5.8 The coherence “negative result” (and why we are proud of it)

The recovery theory needs low coherence. But a dictionary trained for *reconstruction* ends up *coherent*: its atoms overlap a lot ($\mu \approx 0.85$). Plug that into the guarantee and it only certifies exact recovery for $s < 1.1$, far below the $s = 10$ we actually use, and yet separation works fine.

This is not a failure, it is a finding. The guarantees are *worst-case*: they protect against the single most adversarial sparse signal. Real music frames are nowhere near that adversarial, so *average-case* behavior is far better than the worst-case bound. Reporting this sets honest expectations for how much classical theory transfers to learned audio dictionaries, and it is the kind of result reviewers respect because it is easy to hide and we did not.

6 Part 5: Every figure, explained

Figure 1 — the pipeline diagram. Mixture $\rightarrow$ STFT $\rightarrow$ one of four separators $\rightarrow$ soft mask $\rightarrow$ stems. The point of the picture: the four methods are interchangeable parts in one machine, they share a front end and an output and differ only in the middle.

Figure (spectrogram triptych). Three side-by-side spectrograms: the mixture, the true voice, the true band. You can *see* the structure, the voice is a thin set of horizontal harmonic lines (sparse), the band is a denser, more repetitive field (low-rank). This is the visual justification for the whole approach; point at it first.

Figure (RPCA convergence). Two panels. Left: the solver’s error dropping to 10^{-7} in 38 steps, proof the optimization actually converges. Right: the spectrogram’s singular values collapsing after a handful of components, proof the band really is low-rank. Together they say “the assumption is real and the solver finds it.”

Figure (NMF atoms). The learned voice dictionary on top, the band dictionary below, atoms sorted by pitch. Voice atoms cluster in the formant region (where vowels live); band atoms spread lower and wider. This shows the dictionaries learned something musically sensible, not noise.

Figure (results bar chart). The headline table as bars, with the oracle ceilings as dotted lines. The eye immediately sees the staircase, training-free, supervised, deep, and the gap still left to the oracle.

Figure (per-track boxplot). Medians hide spread, so we show the distribution over all 25 tracks. The deep coder’s whole box sits higher, it shifts the full distribution rightward rather than only fixing easy songs, and every method has a long lower tail on dense, vocal-quiet mixtures.

Figure (soft vs binary). A simple bar pair per method showing the soft mask beating the binary mask. Use it to explain why we train toward a soft target.

Figure (training curve). The deep coder’s loss falling while its internal codes stay sparse, evidence the sparsity prior is doing real work throughout training, not switching off.

Figure (oracle gap). Each method as a percentage of the oracle ceiling. The deep coder reaches 54%; the honest message is that masking with the mixture’s phase caps everyone well below 100%, which points at the next research step.

Figure (coherence histogram). The distribution of similarities between dictionary atoms, with the coherence marked far to the right. This is the picture behind the negative result, the atoms are similar, so the worst-case bound is loose.

7 Part 6: Walking the poster, block by block

Stand at the top-left and move down each column. Roughly 4–5 minutes for the whole board; 90 seconds if rushed (do the starred blocks only).

1. **Problem in one sentence***. “Given a finished song, recover the hidden vocal and accompaniment tracks. It is hard because mixing destroys information, so we need a prior.”
2. **Intuition: a song is a picture of sound***. Point at the triptych below it. “Voice is sparse, a few bright harmonic lines; band is low-rank, a repeating pattern. Those are exactly the structures compressed sensing recovers.”
3. **When is recovery guaranteed?** “Three textbook conditions, spark, RIP, coherence, certify you can recover a sparse signal. I will come back to coherence because it bites us later.”
4. **Why music lands in the recoverable regime.** “Concrete numbers: voice is ten atoms out of a hundred, band is rank four to eight. We are inside the guarantees.”
5. **One front end, four separators***. The clean flow diagram. “Everything shares this pipeline; only the boxed middle changes. The starred one is mine.”
6. **Classical recovery stages.** One sentence each on Robust PCA, NMF, K-SVD. “Training-free, then learned templates, then learned alphabets.”
7. **Deep sparse coder***. “A network that is a sparse-recovery solver with its steps unrolled into layers, with the dictionaries learned. Point at the composite-module diagram, fat dictionary lifts, tall dictionary compresses.”
8. **Why unrolling wins.** “Interpretable, small (1.4M params), one forward pass, and it sees a little time context.”
9. **Results***. Point at the bars. “Staircase up to the deep coder: two decibels on voice, eight on the band, best non-oracle on both, and the fastest learned model.”
10. **Ablations.** “Smaller dictionaries generalize better; sparser codes are safer. The knobs behave the way the theory predicts.”
11. **Coherence negative result***. “The honest part: trained dictionaries are coherent, so the worst-case theorem is vacuous, yet it works. Average-case beats worst-case.”
12. **Takeaways***. “One framework unifies four methods; the deep unrolled coder wins; more structure-learning helps monotonically; everything is released.”

8 Part 7: Walking the paper, section by section

- **Abstract / Intro.** States the thesis (these methods are one family) and the contributions (the pipeline, the deep coder, the honest coherence study).

- **Background.** The non-expert on-ramp: STFT, stems, masking, and the fact that the ideal ratio mask is the optimal (Wiener / minimum-mean-square-error) estimator. Cite this section if someone asks “what is a mask, really.”
- **Related work.** Where each method comes from (NMF, RPCA, K-SVD, LISTA, SparseNet) and how we connect them.
- **Preliminaries.** The recovery theory (spark, RIP, coherence, nuclear norm) in compact form.
- **Method.** The shared front end, then the four separators with their update rules, plus the algorithm box for the unrolled solver.
- **Experiments.** Setup, the main table, ablations, soft/binary, fusion, stereo, complexity, and the coherence result.
- **Discussion / Conclusion.** Honest scope (small model, short clips), the phase-masking ceiling, and the next steps (wider time context, complex masks).

9 Part 8: Questions you will get, and answers

“Why not just use Demucs / a big neural net?”

Those win on raw score with millions of times more parameters and data. The point here is the unifying view and the controlled comparison on one front end, plus showing a tiny interpretable model gets surprisingly far. I say this in the discussion explicitly.

“Is the deep model really sparse coding, or just a CNN?”

Really sparse coding: every layer is one FISTA step of a non-negative sparse solve, and the weights are literally dictionaries and shrinkage thresholds. The training curve shows the internal codes stay sparse.

“Your numbers are low.”

On purpose, six-second clips, a 1.4M-parameter model on a laptop GPU, magnitude masking with mixture phase. The contribution is the framework and the comparison, not a leaderboard score, and the oracle-gap figure shows exactly what the phase assumption costs.

“Why does the deep coder beat K-SVD?”

Three reasons: the dictionaries are trained end-to-end for the masking task (not just reconstruction), the fat-then-tall structure adds a clustering effect, and the ± 2 -frame context lets it use a little time information. Removing the context drops it by about half a decibel.

“What is the coherence result really saying?”

That worst-case recovery guarantees do not transfer to trained dictionaries, but average-case performance is fine. It is a caution about over-trusting the bounds, delivered with the measurement to back it up.

“Could this run in real time / in a product?”

The deep coder is one feed-forward pass at 0.12 s per 6 s clip, so yes in principle; the classical solvers are slower because they iterate at test time.

10 Part 9: Glossary

STFT (short-time Fourier transform)

Turns audio into a time-frequency image (the spectrogram).

Magnitude / phase

Loudness vs. waveform alignment; the two parts of the complex spectrogram.

Stem

One of the original tracks (vocals, drums, ...) that were summed into the mix.

Mask

A per-cell number in $[0, 1]$ saying how much of that cell to keep for a source.

Sparse

Mostly zeros, few nonzeros (the voice in frequency).

Low-rank

Built from a few repeating patterns (the band over time).

Dictionary / atom

A learned set of spectral building blocks; one block is an atom.

ℓ_0 / ℓ_1 norm

Counting nonzeros vs. summing magnitudes; ℓ_1 is the convex, solvable stand-in for ℓ_0 .

Nuclear norm

Sum of singular values; the convex stand-in for matrix rank.

Coherence μ

Largest similarity between two dictionary atoms; small is good for recovery.

RIP / spark

Conditions on the measurement matrix that certify sparse recovery.

OMP

Orthogonal matching pursuit, a greedy way to find a sparse code.

NMF

Non-negative matrix factorization, additive templates-and-activations.

K-SVD

An algorithm that learns an overcomplete sparsifying dictionary.

SRC

Sparse-representation classification, label a sample by which class reconstructs it best.

Robust PCA

Splits a matrix into low-rank plus sparse parts.

FISTA / ISTA

Fast iterative shrinkage-thresholding, the proximal solver we unroll.

LISTA

“Learned ISTA,” a network whose layers are unrolled ISTA steps with learned weights.

SI-SDR

Scale-invariant signal-to-distortion ratio, our quality metric, in dB.

IRM / IBM

Ideal ratio / binary mask, the oracle upper bounds.

Oracle

A method given the true answer, used only as a ceiling.

11 Part 10: References and where to look

The full bibliography is in the paper (36 entries). The load-bearing ones: Candès–Li–Ma–Wright (Robust PCA, 2011); Huang et al. (singing-voice RPCA, 2012); Lee–Seung (NMF, 1999/2000); Aharon–Elad–Bruckstein (K-SVD, 2006); Wright et al. (SRC, 2009); Gregor–LeCun (LISTA, 2010); Sun–Nasrabadi–Tran (supervised deep sparse coding, 2019); Le Roux et al. (SI-SDR, 2019); Rafii et al. (MUSDB18, 2017). Code, trained weights, and audio are at github.com/takakhoo/ENGS109_Final_Project.

If you remember three sentences: (1) Music separation is a compressed sensing problem, the voice is sparse and the band is low-rank. (2) Four methods, from a training-free matrix split to a deep unrolled sparse coder, are one family, and the deep one wins (2.0 dB voice, 8.3 dB band, best on both, fastest learned). (3) The honest twist is that the trained dictionaries are too coherent for the worst-case theory, yet average-case performance is fine, so the classical guarantees are a guide, not a guarantee.